

Rapport de projet

Sujet : Développement d'un Assistant Pédagogique

Groupe L3S

Auteurs : Ines Yousfi
Mounir Lamouchi
Rayane Kachbi

Encadrant : David Janiszek

Version du document : 1.0

Confidentialité : Réservé aux étudiants UFR Maths-Info Paris Descartes.

Type de diffusion : Document électronique (.pdf)

Sommaire

1	Introduction	2
1.1	Contexte	2
1.2	Objectif	2
1.3	Méthodologie	2
2	Le Produit : AP-UP	3
2.1	Présentation & Fonctionnalités	3
2.2	Respect du cahier des charges	5
2.3	Technologies utilisées	8
2.3.1	Technologies prévues et utilisées	8
2.3.2	Technologies prévues mais non utilisées	9
2.3.3	Technologies non prévues mais utilisées	9
2.4	Détails sur l'implémentation technique	10
3	Organisation et Suivi de Projet	13
3.1	Planification et Diagramme de Gantt	13
3.2	Analyse du déroulement	14
3.3	Travail Collaboratif	14
4	Retour d'Expérience	15
4.1	Mounir	15
4.2	Ines	16
4.3	Rayane (Chef de projet)	16
5	Débat sur l'IA Générative	18
5.1	Transparence : utilisation des outils d'IA générative	18
5.2	Analyse critique : apport réel des outils et limites pédagogiques	19
6	Compétences Acquis	19
6.1	Compétences techniques	19
6.2	Compétences méthodologiques	20
7	Difficultés rencontrées et solutions apportées	21
8	Pistes d'Amélioration	22
9	Conclusion	23
	Glossaire	24
	Références	24
	Index	26

1 Introduction

1.1 Contexte

L'essor des intelligences artificielles génératives a bouleversé les méthodes de recherche des étudiants. Pour l'institution universitaire, l'usage de ces outils commerciaux tiers soulève de graves questions éthiques, écologiques et de sécurité, notamment les risques de non-conformité RGPD, le pillage de la propriété intellectuelle des enseignants, ainsi que la génération de « bruit » hors programme.

C'est dans ce cadre qu'est né le projet **AP-UP** (Assistant Pédagogique Université Paris Cité), une solution locale basée sur une architecture RAG (*Retrieval-Augmented Generation*).

Le présent document constitue le rapport final de ce projet. Il a pour but de formaliser la livraison de l'application, d'en synthétiser la réalisation technique et de dresser un bilan critique de l'ensemble du travail accompli en se basant sur les objectifs initiaux définis dans les documents rédigés pendant la phase de pré-développement. Il donne également un retour personnel de l'équipe du projet.

1.2 Objectif

L'objectif principal est de concevoir un assistant intelligent capable de fournir des réponses fiables et strictement limitées au périmètre des documents fournis par les enseignants.

À travers les différents chapitres de ce rapport, nous montrons comment la solution finale atteint les exigences initiales du projet, à savoir :

- **Réduction des hallucinations** : Grâce aux cours des enseignants, le système évite d'inventer des informations. Il informe également l'utilisateur s'il ne trouve pas la réponse dans le cours.
- **Traçabilité** : Chaque réponse est accompagnée de ses sources exactes (nom du document, page, section), permettant à l'étudiant de vérifier l'information ou de trouver les sections intéressantes des cours.
- **Performance interactive** : Garantir un temps de réponse compatible avec une utilisation fluide.
- **Modularité** : Permettre l'évolution technique du système, la migration de la base de données, ou le remplacement des modèles de langage selon les besoins futurs.

1.3 Méthodologie

Le projet s'est déroulé sur une période de 12 semaines, articulée autour de deux grandes phases : la conception et le développement. Cette démarche a été segmentée en cinq étapes clés :

1. **Phase d'analyse et de recherches** : Étude des technologies, étude de marché, recherches sur les différents concepts (RAG, LLM, Hallucinations...) pour une meilleure compréhension de l'objectif et la définition des axes/modules principaux (ingestion, RAG, données, filtres, interface, API).

2. **Phase de conception** : Rédaction des documents structurants (cahier des charges, cahier de recettes) et modélisation de l'architecture logicielle via des schémas UML (conception générale et détaillée).
3. **Phase de développement** : Implémentation du pipeline complet, incluant l'extraction de texte, le découpage en chunks, la vectorisation, l'intégration du LLM et la création de l'interface conversationnelle.
4. **Phase de tests et validation** : Vérification du respect des critères d'acceptabilité, tests unitaires sur les modules et évaluation des performances de l'application web.
5. **Phase de documentation et livraison** : Rédaction des manuels utilisateur et installation, production du rapport final.

2 Le Produit : AP-UP

AP-UP est un assistant pédagogique conversationnel local, hébergé sur les serveurs de l'université (Pléiade). Il permet aux étudiants de poser des questions sur leurs cours et d'obtenir des réponses précises, sourcées et ancrées dans les documents pédagogiques déposés par leurs enseignants. L'application est accessible depuis n'importe quel navigateur web moderne (Chrome, Firefox, Edge, Brave).

2.1 Présentation & Fonctionnalités

L'application distingue deux types d'utilisateurs et propose deux espaces distincts bien définis.

Espace étudiant

Après connexion, l'étudiant accède à l'espace principal composé d'une barre latérale listant les cours disponibles, d'une zone de conversation avec l'assistant et d'un champ de saisie pour poser des questions.

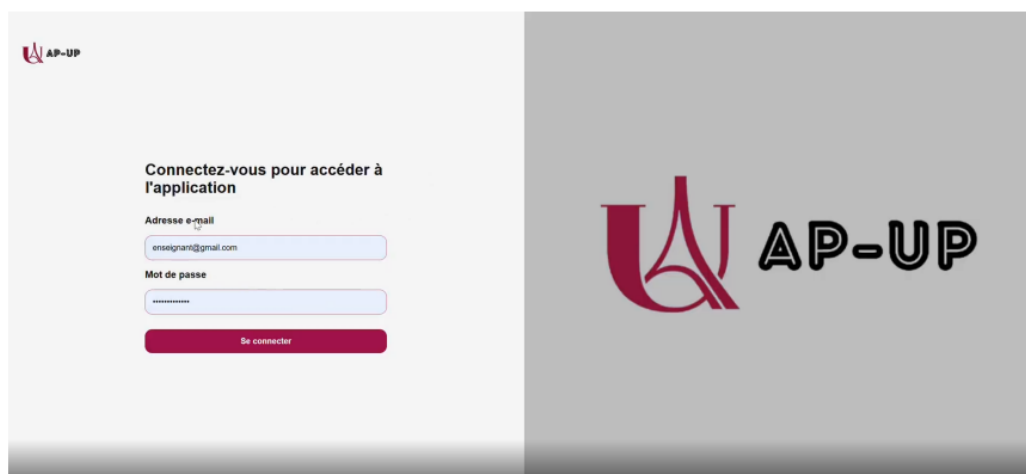


FIGURE 1 – Interface de la page de connexion

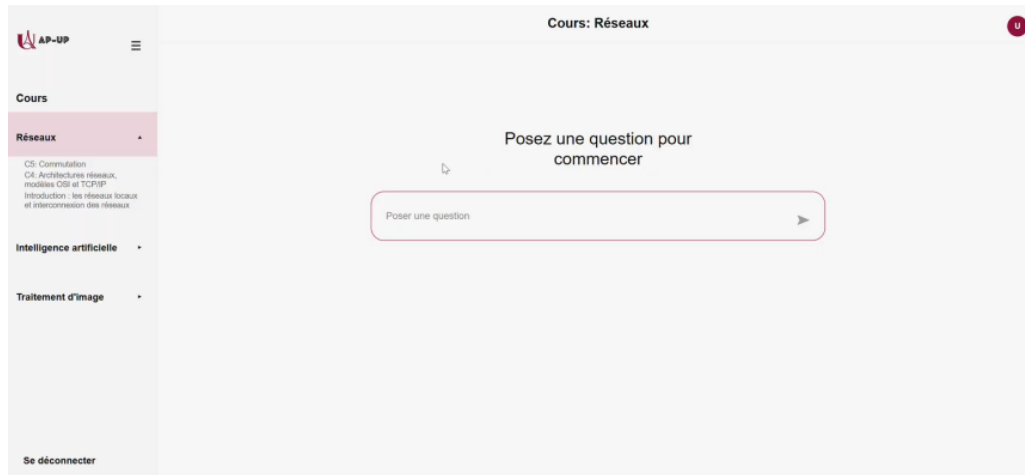


FIGURE 2 – Vue de l'espace principal de l'étudiant

Pour interagir avec l'assistant, l'étudiant sélectionne un cours dans la barre latérale puis saisit sa question. Le système recherche les passages les plus pertinents dans les documents du module via le moteur RAG, puis génère une réponse affichée directement dans la zone de conversation.

L'assistant répond uniquement à partir des documents du module sélectionné. Si la question ne correspond pas au contenu du cours, elle est systématiquement refusée par le système de filtrage avec un message explicatif adapté.

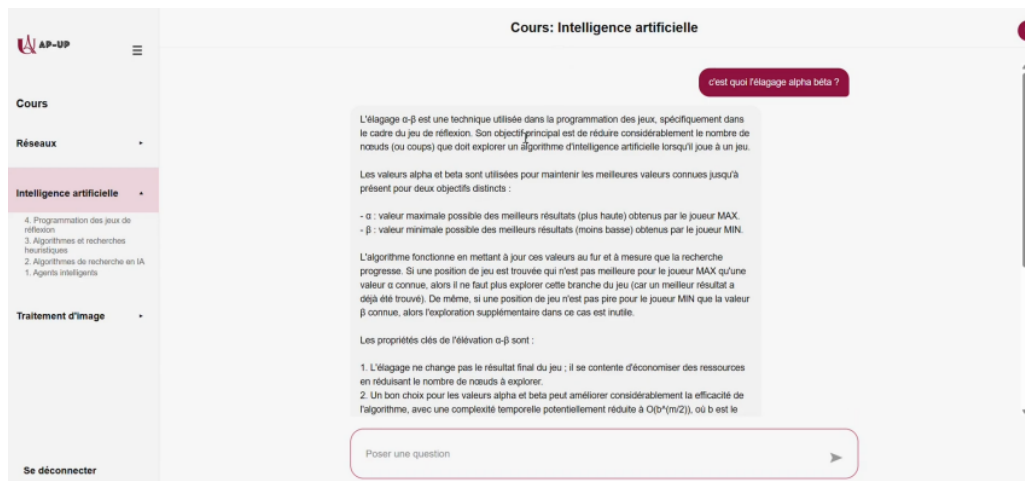


FIGURE 3 – Exemple d'échange conversationnel avec l'assistant

Espace administrateur (enseignant)

Un utilisateur ayant le rôle enseignant peut accéder au tableau de bord d'administration depuis le menu profil en cliquant sur « Administration professeur ». Ce tableau de bord propose deux onglets distincts :

- **Gestion des documents** : L'enseignant peut y sélectionner un cours, nommer et importer un document, consulter la liste des documents déjà indexés et en suppri-

mer si nécessaire. Les formats supportés sont le PDF, DOC/DOCX, PPT/PPTX, MP3 et MP4. Lors de l'importation, le backend extrait le contenu, le découpe en fragments, calcule les représentations vectorielles et enregistre les données dans la base.

- **Prompt système** : L'enseignant peut y définir les instructions spécifiques données à l'assistant pour un cours précis, par exemple en précisant le niveau de détail attendu, le ton à adopter ou les limites strictes à respecter.

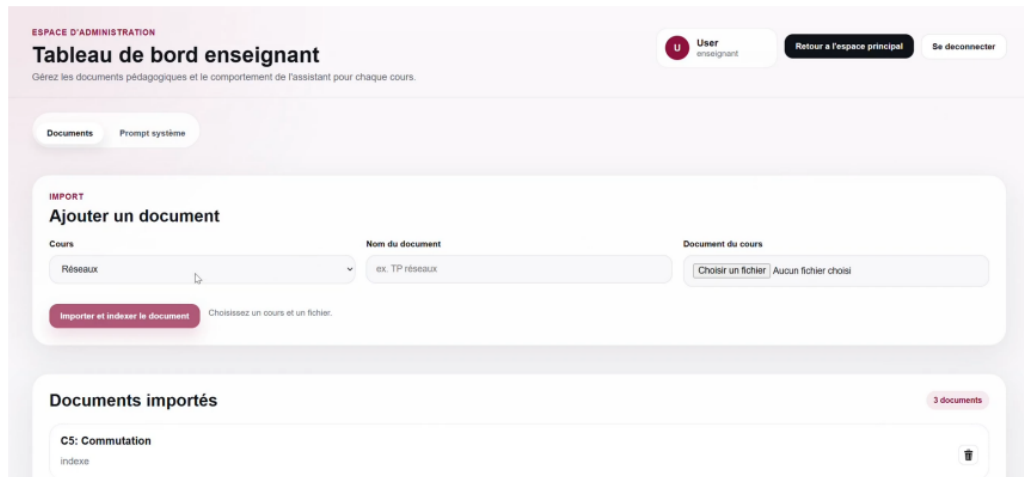


FIGURE 4 – Tableau de bord enseignant — Onglet de gestion des documents

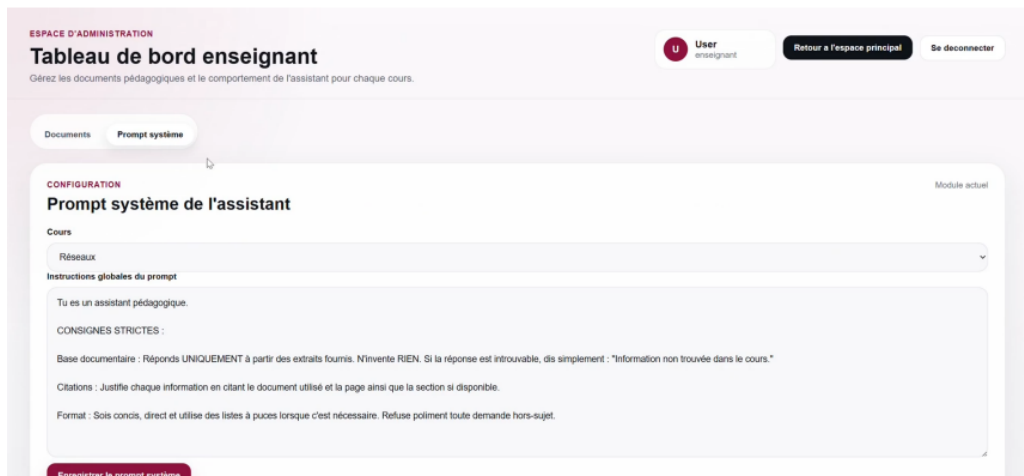


FIGURE 5 – Tableau de bord enseignant — Onglet de configuration du prompt système

2.2 Respect du cahier des charges

Tableau de conformité : Reprise du tableau des fonctionnalités du cahier des charges avec la colonne de validation où tout ce qui a été réalisé a été coché.

Fonctionnalités principales

ID	Description	Contraintes	Dép.	Action (Procédure)	Résultat attendu	Validé
FP-01	Accès à l'application web	Navigateur compatible (Chrome, Firefox)	Aucune	Ouvrir l'adresse web de l'application.	L'interface se charge correctement sans erreur.	[X]
FP-02	Authentification utilisateur / administrateur	Compte valide	FP-01	Entrer les identifiants et valider.	L'utilisateur / administrateur est connecté et accède à son espace.	[X]
FP-03	Dépôt de documents pédagogiques	Fichier valide	FP-02	Aller dans l'espace administrateur et uploader un fichier.	Le document est accepté et ajouté à la liste des fichiers.	[X]
FP-04	Indexation automatique des documents	Document correctement uploadé	FP-03	Vérifier le statut du document importé.	Le système découpe le document, génère les embeddings et les stocke dans la base de données vectorielle.	[X]
FP-05	Pose de question par un étudiant	Interface de chat fonctionnelle	FP-02	Écrire une question dans la barre de chat et l'envoyer.	La question s'affiche et l'interface indique que le modèle « réfléchit ».	[X]
FP-06	Recherche sémantique	Base pgvector active	FP-05	Soumettre une question spécifique au cours.	Le système identifie et récupère en arrière-plan les passages pertinents.	[X]
FP-07	Génération de réponse par le modèle	API Pléiade / Ollama active	FP-06	Attendre la fin du processus de génération.	L'assistant formule une réponse cohérente basée sur les documents.	[X]
FP-08	Filtrage de sécurité des requêtes	Système de filtrage / SLM actif	FP-05	Poser une question hors-sujet ou inappropriée.	La requête est filtrée ou bloquée, un message indique que l'IA ne répond qu'au cours.	[X]
FP-09	Affichage de la réponse	Interface Chainlit fonctionnelle	FP-07	Observer le retour de l'assistant suite à la question posée.	La réponse s'affiche correctement dans la conversation.	[X]

TABLE 1 – Matrice de conformité des fonctionnalités principales

Fonctionnalités secondaires

ID	Description	Contraintes	Dép.	Action (Procédure)	Résultat attendu	Validé
FS-01	Gestion des rôles	Comptes configurés	FP-02	Se connecter successivement avec un compte Étudiant puis Enseignant.	L'étudiant n'a accès qu'au chat ; l'enseignant a accès au panneau d'administration.	[X]
FS-02	Mise en cache des réponses	Système de cache actif	FP-07	Poser une question, chronométrer, puis reposer exactement la même question.	La seconde réponse est envoyée plus rapidement.	[]
FS-03	Suppression d'un document	Document existant	FP-03	Depuis l'espace administrateur, sélectionner un document, cliquer sur « Supprimer ».	Le document et ses vecteurs (embeddings) associés disparaissent du système.	[X]
FS-04	Gestion des erreurs de saisie	Interface active	FP-05	Envoyer une requête vide ou avec uniquement des espaces.	Le système refuse l'envoi et demande de formuler une vraie question.	[X]
FS-05	Affichage des sources utilisées	Traçabilité des passages active	FP-07	Inspecter la réponse générée par l'IA.	L'interface affiche en bas de la réponse les références exactes utilisées (nom du PDF, page).	[X]

TABLE 2 – Matrice de conformité des fonctionnalités secondaires

Fonctionnalités facultatives

ID	Description	Contraintes	Dép.	Action (Procédure)	Résultat attendu	Validé
FF-01	Export des réponses	Fonctionnalité d'export disponible	FP-09	Cliquer sur le bouton d'export à côté d'une réponse.	La réponse est téléchargée sur l'ordinateur de l'utilisateur.	[]
FF-02	Mise en cache globale (interutilisateurs)	Système de cache global actif, 2 comptes distincts	FP-07	Étudiant A pose une question. Depuis un autre navigateur, Étudiant B pose exactement la même question.	La réponse est fournie instantanément à l'Étudiant B sans nouvel appel au modèle IA.	[]

TABLE 3 – Matrice de conformité des fonctionnalités facultatives

2.3 Technologies utilisées

Plusieurs technologies ont été étudiées, testées puis intégrées selon les besoins rencontrés au cours de la réalisation. Certaines technologies prévues initialement ont été conservées tout au long du projet, d'autres ont finalement été abandonnées, tandis que de nouvelles solutions ont été ajoutées afin de répondre à des contraintes techniques apparues pendant le développement.

2.3.1 Technologies prévues et utilisées

Plusieurs technologies prévues dès la phase de conception ont été utilisées dans la version finale du projet.

Gestion des données et architecture back-end

- **PostgreSQL** : Le projet repose sur PostgreSQL pour le stockage des données relationnelles, notamment les utilisateurs, les documents, les modules, les chunks et l'historique des conversations.
- **pgvector** : L'extension pgvector a été intégrée à PostgreSQL afin de permettre le stockage et la recherche vectorielle directement dans la base de données, ce qui a permis de centraliser les données relationnelles et vectorielles dans une seule infrastructure.
- **SQLAlchemy** : Afin de manipuler la base de données de manière structurée, SQLAlchemy a été utilisé comme ORM. Cette solution permet de définir les modèles de données directement en Python et de simplifier les interactions avec la base de données.
- **Python** : Le back-end a principalement été développé en Python, qui constitue le langage central du projet.

Module RAG et intelligence artificielle

- **sentence-transformers** : Pour le système RAG, la bibliothèque sentence-transformers a été utilisée afin de générer les embeddings vectoriels des documents et des requêtes utilisateur. Ces vecteurs permettent d'effectuer des comparaisons sémantiques entre les questions posées et les contenus pédagogiques stockés dans la base.
- **OpenAI** : La bibliothèque OpenAI a été utilisée pour interagir avec certains modèles de langage lors des phases de développement et d'expérimentation.
- **Ollama** : Le projet utilise également Ollama afin d'exécuter localement plusieurs modèles de langage hébergés sur le serveur Pléiade de l'université.
- **tiktoken** : La bibliothèque tiktoken a été intégrée afin de gérer le comptage des tokens lors des interactions avec les modèles de langage.

Traitement documentaire et ingestion

- **pdfplumber & Unstructured** : Le traitement des documents PDF repose principalement sur pdfplumber pour l'extraction de texte et sur Unstructured pour le parsing documentaire open source.
- **Whisper** : Pour le traitement multimédia, Whisper (openai-whisper) a été utilisé afin de réaliser la transcription automatique des fichiers audio et vidéo.

Tests et validation

- **pytest & unittest** : Les bibliothèques pytest et unittest ont été utilisées pour développer et organiser les tests unitaires des différents modules.

2.3.2 Technologies prévues mais non utilisées

Certaines technologies envisagées au début du projet n'ont finalement pas été retenues, principalement pour des raisons de performance, de flexibilité ou de contraintes de temps.

Chainlit Chainlit avait initialement été envisagé pour développer rapidement une interface conversationnelle dédiée aux applications d'intelligence artificielle. Cependant, après plusieurs tests, cette solution s'est révélée trop limitée pour les besoins du projet. L'interface proposée manquait de flexibilité, notamment pour gérer plusieurs espaces distincts comme une page de connexion, un espace étudiant et un espace enseignant. Des essais d'intégration avec React ont également été réalisés, mais Chainlit imposait toujours une structure trop rigide.

Marker La bibliothèque Marker avait été étudiée pour le traitement avancé de fichiers PDF complexes. Malgré des résultats intéressants, cette technologie nécessitait des ressources matérielles importantes, notamment en mémoire et en puissance de calcul. Elle a donc été abandonnée au profit de solutions plus légères et mieux adaptées à l'environnement du projet.

Playwright L'utilisation de Playwright était prévue afin d'implémenter un module de web scraping destiné à récupérer automatiquement certains contenus pédagogiques. Cette fonctionnalité n'a cependant pas pu être développée dans les délais impartis et la technologie n'a donc pas été intégrée dans la version finale du projet.

Modèle llama3.2 Le modèle llama3.2 avait initialement été envisagé comme modèle « juge » dans le module de filtrage. Cependant, les tests réalisés ont montré que ce modèle ne respectait pas suffisamment les consignes strictes imposées par le système, notamment l'obligation de répondre uniquement par « OUI » ou « NON ». Malgré plusieurs ajustements de prompts, les réponses générées restaient parfois imprévisibles et difficiles à exploiter automatiquement.

2.3.3 Technologies non prévues mais utilisées

Au cours du développement, plusieurs technologies non envisagées initialement ont été intégrées afin de répondre à de nouveaux besoins techniques ou pour améliorer les performances du projet.

API et communication front-end / back-end L'évolution du projet vers une architecture séparant clairement le front-end et le back-end a nécessité l'intégration de **FastAPI** afin de développer une véritable API REST.

Le serveur **Uvicorn** a été utilisé pour exécuter l'application FastAPI, tandis que **python-multipart** a été ajouté afin de gérer l'envoi de fichiers depuis l'interface utilisateur.

Interface utilisateur L'interface finale a été développée avec **React** et **Vite** en JavaScript. Ce choix a permis de concevoir une interface plus flexible et modulaire que celle proposée par Chainlit.

La navigation entre les différentes pages a été gérée avec **React Router**. Les échanges entre le front-end et le back-end se font via des requêtes HTTP utilisant **fetch**, avec des données échangées au format JSON.

Traitement documentaire avancé Plusieurs bibliothèques ont été ajoutées pour améliorer le traitement de certains documents PDF complexes :

- **docling**.
- **pymupdf**.
- **pypdf**.

Ces solutions ont permis d'obtenir de meilleurs résultats lors des phases de tests et d'améliorer la gestion des différents formats documentaires.

Sécurité et déploiement La bibliothèque **bcrypt** a été intégrée afin de sécuriser le hachage des mots de passe utilisateurs. Le projet a également été conteneurisé avec **Docker**, facilitant le déploiement et la reproductibilité de l'environnement de développement.

Modèle ministral-3 :3b Après plusieurs tests comparatifs entre différents modèles disponibles sur le serveur Pléiade, le modèle ministral-3 :3b a été retenu pour le module de filtrage. Ce modèle s'est révélé être le meilleur compromis entre légèreté, rapidité et respect des consignes de réponse binaire imposées par le système.

2.4 Détails sur l'implémentation technique

Module d'ingestion et moteur RAG

Description du pipeline : Une architecture d'ingestion dynamique a été développée autour d'un orchestrateur principal (**DocumentProcessor**) chargé de détecter le type de fichier et d'analyser la complexité des PDF.

Le système route intelligemment les documents vers des parsers spécifiques : les PDF strictement textuels sont traités de manière légère par **pdfplumber** ; les PDF complexes sont envoyés vers **Docling** ; les documents scannés ou sous formats Office/HTML sont gérés par la librairie **unstructured** avec un module d'OCR intégré ; et les fichiers audio/vidéo (MP4, MP3) sont transcrits automatiquement par le modèle **Whisper**.

Côté inférence, le cœur du système RAG orchestre la recherche sémantique et la génération de la réponse. La recherche (*Retriever*) utilise un modèle **SentenceTransformer** pour encoder la question de l'utilisateur et récupérer les passages les plus pertinents dans la base vectorielle.

Ces passages sont ensuite transmis au **PromptBuilder**, qui assemble un contexte riche pour le LLM. Enfin, le moteur (**RAGEngine**) interroge le serveur d'inférence et renvoie la réponse sous forme de flux continu (*streaming*, token par token).

Explication des choix techniques : La robustesse du système RAG repose avant tout sur la qualité des données extraites. Afin de préserver le sens du cours pour le LLM, nous avons abandonné l'idée d'un découpage purement mathématique (par taille fixe) qui coupait régulièrement des phrases ou des concepts en deux.

À la place, une stratégie de découpage hybride a été implémentée. Le texte est d'abord découpé selon sa structure sémantique en exploitant les balises Markdown (titres `##`) générées par `Docling`, puis par paragraphes. Le redécoupage par taille stricte de tokens n'intervient qu'en dernier recours si un bloc sémantique est trop long.

Chaque fragment (*chunk*) est ensuite enrichi avec des métadonnées précises (nom du fichier, numéro de page, section) pour permettre au modèle de citer ses sources avec exactitude. Un chevauchement (*overlap*) est également appliqué pour éviter la perte de contexte aux frontières des fragments.

Exemples de tests menés : Lors de nos premiers tests, nous avons constaté que les extracteurs PDF classiques échouaient face à des documents contenant des tableaux, des doubles colonnes ou des formules mathématiques. Le choix de `Docling` a été fait après avoir testé d'autres solutions (comme `Marker`), car `Docling` gère nativement la conversion en Markdown structuré tout en consommant beaucoup moins de mémoire RAM.

Lors de nos phases de test, nous avons également remarqué que l'assistant perdait le fil si l'étudiant posait des questions contenant des anaphores (par exemple : « *Peux-tu m'expliquer ce concept ?* »).

Pour pallier cela, le constructeur de prompt injecte désormais les échanges récents de l'utilisateur en plus d'une instruction explicite demandant au modèle d'utiliser cet historique pour résoudre les références floues.

Module de base de données et filtrage

Description du pipeline : La base de données a été mise en place avec PostgreSQL et l'extension pgvector. Le fichier `models.py` définit les tables de l'application (utilisateurs, modules, documents, chunks et historique) et le fichier `vector_store.py` centralise toutes les fonctions d'accès aux données.

Lorsqu'une question est soumise par l'étudiant, elle passe d'abord par un filtre d'entrée. Le modèle `ministral-3.3b` reçoit la question ainsi que des extraits du cours et répond par OUI ou NON selon si la question est en lien avec le contenu.

Une fois la réponse générée, elle passe par un filtre de sortie. La similarité cosinus est d'abord calculée entre la réponse et les documents utilisés. Si le score est suffisamment élevé, la réponse est acceptée directement. Si le score est trop faible, elle est bloquée.

Lorsqu'une question est bloquée, le système propose automatiquement trois questions alternatives reformulées à partir des passages les plus proches dans les documents du cours, afin de guider l'étudiant vers une question plus adaptée.

Explication des choix techniques : La table `chunks` stocke à la fois le texte des fragments et leurs vecteurs sémantiques en 768 dimensions, ce qui permet d'effectuer des

recherches par similarité directement dans la base vectorielle.

Le module de filtrage a été développé pour garantir que l'assistant répond uniquement à des questions en lien avec le cours et que les réponses générées sont bien ancrées dans les documents.

Il repose sur une logique hybride combinant la similarité cosinus et un petit modèle de langage utilisé comme juge. Un mécanisme de secours (*fallback*) basé sur la similarité cosinus a également été intégré pour prendre le relais si le modèle est indisponible.

Exemples de tests menés : Le modèle `ministral-3:3b` a été retenu après plusieurs tests comparatifs ; `llama3.2` et `phi4-mini` ont été abandonnés car ils ne suivaient pas de manière fiable les instructions strictes et produisaient du texte libre au lieu d'une réponse binaire.

Pour les cas intermédiaires, appelés « zone grise », le modèle `ministral-3:3b` est à nouveau sollicité pour vérifier si la réponse est fidèle aux extraits du cours.

Ce mécanisme a été ajouté après avoir constaté lors des tests que certaines réponses incorrectes passaient sans être détectées avec le calcul de la similarité cosinus seul.

Interface utilisateur (Front-end) et API

Description du pipeline : L'application est organisée autour de plusieurs pages principales : une page de connexion, un espace principal principalement dédié aux étudiants pour utiliser le chat et un espace enseignant pour la gestion des documents pédagogiques. L'interface de chat permet à l'étudiant de saisir une question, de l'envoyer au système, puis d'afficher la réponse générée.

Une API a été mise en place avec FastAPI. L'intégration a consisté à relier les actions effectuées depuis l'interface aux traitements du back-end.

Par exemple, lorsqu'un étudiant pose une question, le front-end envoie la requête à l'API, qui la transmet ensuite au système RAG. Le back-end effectue alors le filtrage, la recherche des passages pertinents, puis la génération de la réponse.

Le résultat est ensuite renvoyé à l'interface et affiché dans la conversation. Pour l'espace enseignant, le même principe est appliqué au dépôt et à la suppression de documents.

Explication des choix techniques : La partie front-end a été développée avec React afin de proposer une interface modulable et structurée de manière semblable à celle des frameworks modernes. Cette organisation permet de séparer clairement les usages selon le rôle de l'utilisateur et de rendre l'application plus lisible.

Une vérification est également effectuée du côté de l'interface afin d'éviter l'envoi de questions vides.

L'API sert d'intermédiaire entre les actions réalisées par l'utilisateur et les traitements effectués côté serveur. Les échanges entre le front-end et l'API se font via des requêtes HTTP avec des données formatées au format JSON.

Exemples de tests menés : Des tests ont été réalisés directement depuis l'interface pour vérifier le fonctionnement complet de l'application, notamment l'envoi de questions, l'affichage des réponses, les comportements et les erreurs face aux questions vides, les questions hors sujet ainsi que la gestion globale des documents.

3 Organisation et Suivi de Projet

3.1 Planification et Diagramme de Gantt

Au début du projet, l'équipe a établi une planification prévisionnelle afin d'organiser les différentes étapes nécessaires au développement de l'assistant pédagogique AP-UP. Le projet a été structuré autour de cinq grandes phases :

- une phase d'analyse et de recherches ;
- une phase de conception ;
- une phase de développement et tests ;
- une phase de tests et validation ;
- une phase de documentation et livraison.

Cette organisation a permis de répartir progressivement les tâches entre les membres du groupe tout en conservant une vision globale de l'avancement du projet.

Afin de suivre l'évolution du développement, nous avons utilisé un diagramme de Gantt permettant de visualiser les différentes étapes du projet ainsi que les échéances associées.

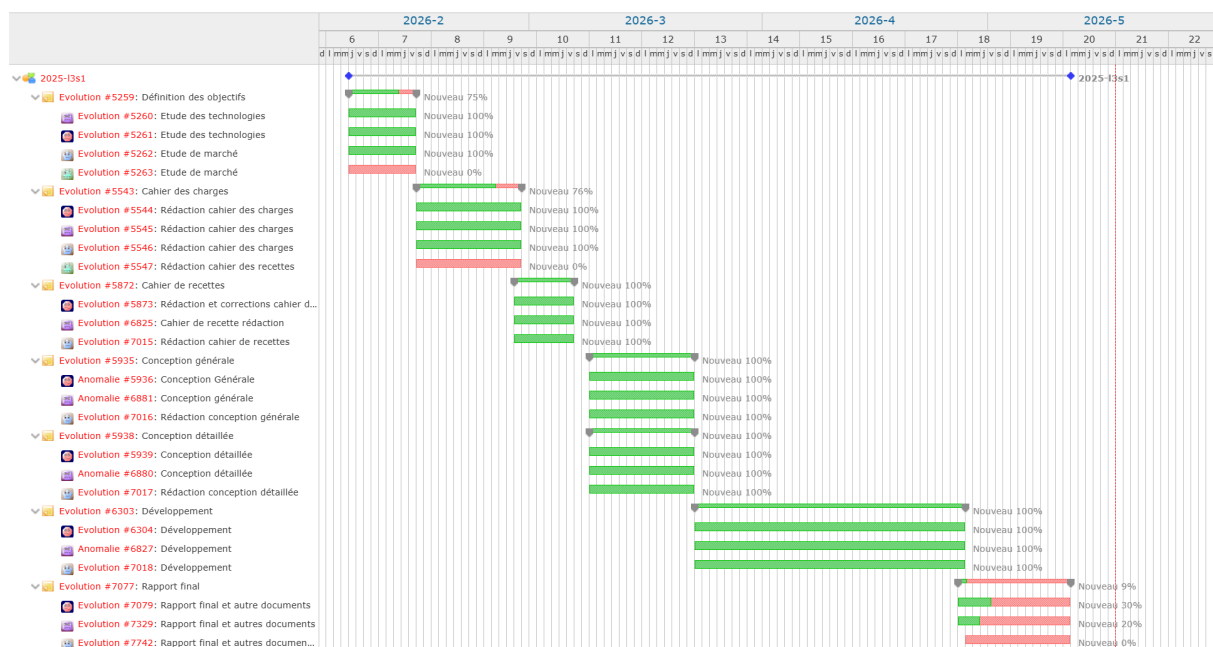


FIGURE 6 – Diagramme de Gantt du projet

3.2 Analyse du déroulement

Dans l'ensemble, le projet a suivi les grandes phases définies dans le planning initial. Certaines étapes ont toutefois demandé davantage de temps que prévu, notamment durant la phase de conception et d'organisation du projet.

Nous avons notamment dû nous adapter à plusieurs changements au cours du semestre, comme le démarrage du projet avec un léger retard ainsi que le départ d'un membre du groupe. Ces éléments ont conduit à prolonger légèrement la phase de conception avant le début du développement principal.

Malgré cela, nous avons rapidement réorganisé la répartition des tâches afin de maintenir l'avancement du projet et respecter les objectifs principaux fixés au départ.

Une partie importante du travail concernait les choix techniques et l'intégration des différentes technologies utilisées dans le projet. La mise en place de l'architecture globale de l'application, la communication entre les différents composants ainsi que l'adaptation des outils retenus au contexte du projet ont demandé plusieurs phases d'ajustement au cours du développement.

Nous avons également dû consacrer du temps aux phases de tests, de correction et d'amélioration progressive de l'application afin d'obtenir une version suffisamment stable et cohérente pour le rendu final.

Certaines fonctionnalités secondaires prévues initialement, comme des aspects avancés de la gestion des profils utilisateurs, n'ont cependant pas pu être entièrement finalisées dans les délais du projet.

Tout au long du semestre, nous avons su adapter notre organisation afin de continuer à avancer efficacement malgré les ajustements nécessaires. Finalement, nous avons réussi à respecter les principales échéances du projet et à produire une application fonctionnelle accompagnée des différents livrables demandés.

3.3 Travail Collaboratif

Le travail collaboratif du projet AP-UP a été organisé autour d'un dépôt SVN hébergé sur la forge universitaire : <https://forge.ens.math-info.univ-paris5.fr/svn/2025-13s1>.

L'utilisation de SVN a permis à l'équipe de centraliser le code source du projet, de suivre les différentes modifications effectuées au cours du développement et de faciliter le travail simultané des membres du groupe.

Le dépôt était organisé selon une structure classique composée des dossiers Branches, Trunk et Tags :

- **Branches** : chaque membre du groupe dispose de sa propre branche de développement nominative. Cela permettait de développer et tester des fonctionnalités sans modifier directement la version principale du projet et éviter les problèmes.

- **Trunk** : le trunk contenait les différentes versions stables et exécutables de l'application. Les versions V1, V2 et V3 y ont été intégrées progressivement au fur et à mesure de l'avancement du projet.
- **Tags** : le tag était utilisé pour soumettre la version finale du code source.

Cette organisation a permis de maintenir une architecture claire tout en facilitant l'intégration progressive des différentes fonctionnalités développées.

L'équipe a adopté une méthode de travail inspirée des méthodes agiles. Les échanges quotidiens étaient réalisés principalement via WhatsApp afin de communiquer rapidement sur les tâches en cours, les problèmes rencontrés, et les décisions techniques importantes.

Des réunions hebdomadaires étaient également organisées afin de suivre l'avancement du projet avec et sans notre professeur encadrant, répartir les nouvelles tâches et définir les objectifs des semaines suivantes.

4 Retour d'Expérience

4.1 Mounir

Ce projet a été une expérience vraiment enrichissante. C'est la première fois que je travaillais sur un projet d'une telle envergure, avec une vraie architecture backend, une base de données vectorielle et des modèles de langage locaux. Au début, j'avoue que certains concepts comme les embeddings, la similarité cosinus ou le SLM-as-a-judge me semblaient abstraits. En les implémentant concrètement, tout est devenu beaucoup plus clair.

La partie qui m'a le plus appris est le module de filtrage. J'ai notamment beaucoup galéré avec llama3.2 au départ — le modèle était censé répondre uniquement OUI ou NON, mais il produisait régulièrement du texte libre, des explications, parfois même des questions en retour. Ça rendait le parsing impossible et le filtrage complètement non fonctionnel. J'ai passé beaucoup de temps à adapter le prompt, tester différentes formulations, avant de finalement changer de modèle. Ce genre de blocage est frustrant sur le moment, mais ça m'a appris à ne pas m'entêter sur une solution qui ne fonctionne pas et à savoir pivoter.

J'ai aussi découvert un bug assez tardivement — certains chunks issus de tableaux ou de listes n'avaient pas de section détectée et affichaient None comme métadonnée. Ce n'est pas un bug critique, mais ça affectait l'affichage des sources pour l'étudiant. Le corriger m'a demandé de comprendre en profondeur le fonctionnement du chunking, ce qui au final m'a permis de mieux maîtriser cette partie du projet.

J'ai aussi réalisé l'importance des tests unitaires : sans eux, chaque modification pouvait casser quelque chose sans qu'on s'en rende compte immédiatement.

Sur le plan humain, ce projet ne s'est pas toujours déroulé sans accroc. Le départ d'un membre en cours de semestre a été un moment difficile à gérer, car il a fallu absorber certaines tâches supplémentaires sans vraiment avoir anticipé cette situation. Ajouter ça à des disponibilités qui n'étaient pas toujours compatibles entre nous, et certaines semaines devenaient vraiment chargées. Mais avec du recul, ça m'a appris à être plus adaptable et à ne pas compter uniquement sur l'organisation initiale pour avancer.

Si c'était à refaire, je passerais plus de temps en amont sur le choix des modèles pour éviter plusieurs allers-retours. Mais globalement je suis fier du résultat et de ce qu'on a réussi à construire ensemble.

4.2 Ines

Ce projet a été une expérience très enrichissante pour moi, car j'ai pu travailler sur un projet concret qui m'intéressait vraiment. Il m'a permis de mieux comprendre comment une application complète est construite, depuis l'interface utilisateur jusqu'à la communication avec le back-end. Le fait de travailler sur le front-end, l'API et l'intégration m'a donné une vision plus globale du fonctionnement d'un projet logiciel.

Le projet m'a aussi permis de découvrir et d'utiliser de nouveaux outils comme React, Vite et FastAPI. J'ai appris à créer une interface plus interactive, à organiser plusieurs pages et à gérer les échanges entre le front-end et le back-end. Cela m'a aidé à mieux comprendre le lien entre ce que voit l'utilisateur et ce qui est traité côté serveur. Comme la majorité des technologies utilisées étaient nouvelles pour moi, j'ai dû apprendre en parallèle plusieurs aspects du développement web moderne, notamment la création d'interfaces avec React, la gestion des appels API et l'intégration entre les différentes parties de l'application. Cela m'a demandé une importante phase d'adaptation mais m'a permis de progresser rapidement et de gagner en autonomie.

Au début, j'ai aussi pas mal galéré avec Chainlit lorsqu'on essayait de construire l'interface. Certaines choses ne fonctionnaient pas comme on le voulait et ça nous a fait perdre du temps au début du projet. Sur le moment c'était assez frustrant, mais avec du recul ça m'a permis de mieux comprendre les limites de certains outils et l'importance de choisir des technologies adaptées à ce qu'on veut réellement développer.

Ce projet m'a aussi amenée à m'intéresser à des concepts que je ne connaissais pas forcément au départ, comme le RAG, l'ingestion de documents, l'indexation ou encore la recherche dans les documents du cours. Même si je n'ai pas travaillé directement sur toutes ces parties, j'ai dû comprendre leur rôle pour pouvoir intégrer correctement l'interface avec le reste de l'application. En testant certaines fonctionnalités de mon côté, j'ai mieux compris comment les différents modules communiquent entre eux.

Sur le plan humain, le projet n'a pas toujours été simple à organiser. Il y a eu des difficultés de communication dans le groupe et le départ d'un membre en cours de semestre, ce qui nous a obligés à revoir certaines tâches et à nous adapter. Cette expérience m'a appris à être plus autonome, à chercher des solutions par moi-même et à mieux gérer les blocages pendant le développement.

4.3 Rayane (Chef de projet)

Si je devais résumer cette expérience de projet en une image, je dirais qu'elle ressemblait à une courbe de cosinus : une succession de hauts et de bas. Malgré les difficultés rencontrées, ce projet a été éducatif, aussi bien sur le plan technique que sur le plan humain et organisationnel.

D'un point de vue technique, j'ai eu l'opportunité de découvrir plusieurs technologies qui

correspondent directement à mes objectifs professionnels : RAG, l'utilisation de modèles locaux avec Ollama, ainsi que les bases de données vectorielles. Ces sujets m'intéressaient déjà avant le projet, mais le fait de devoir les mettre en pratique dans une application concrète m'a permis de réellement comprendre leur fonctionnement, leurs limites et les choix d'architecture associés.

Cependant, l'aspect le plus marquant de cette expérience a été mon rôle de chef de projet tout au long du semestre. Au départ, je pensais que les principales difficultés seraient essentiellement techniques. Finalement, la partie la plus exigeante a surtout été la gestion du groupe, l'organisation du travail et le maintien d'une progression régulière malgré les contraintes rencontrées.

Dès les premières semaines, j'ai compris que nous risquions de prendre du retard par rapport aux autres groupes. Cette impression s'est rapidement confirmée en raison d'un encadrement plus limité que prévu : contrairement aux autres équipes, nous n'avons pas eu de réunion lors de la première semaine, et nous n'avons finalement pu réaliser que 7 réunions sur les 12 initialement prévues. À cela s'ajoutaient le manque de réponses régulières à mes mails ainsi que des retours parfois difficiles à appliquer compte tenu de l'avancement déjà pris par le projet.

Par ailleurs, la communication au sein du groupe n'était pas toujours simple, notamment à cause des disponibilités différentes de chacun. La situation a également été compliquée par l'abandon d'un membre du groupe dès le début du projet.

Face à cette situation, j'ai essayé d'assurer une stratégie de travail aussi claire et structurée que possible. Je me suis occupé de la répartition des tâches, du suivi de l'avancement des différents membres, ainsi que de la relecture et de la cohérence globale du travail produit. J'ai également dû prendre certaines décisions techniques et organisationnelles afin de maintenir une évolution continue, une direction claire pour le projet et de ne pas dépasser les différentes deadlines. La communication n'était pas notre point fort, mais j'ai essayé de compenser cela en centralisant régulièrement les informations importantes et les consignes sur notre espace d'échange.

Concernant le développement, je me suis principalement occupé des modules de RAG et d'ingestion de données. Cette partie a été particulièrement délicate, car ces composants constituaient la base nécessaire au reste de l'équipe pour pouvoir tester et développer les autres fonctionnalités du projet. J'ai donc dû livrer certaines parties rapidement, parfois avant d'avoir pu les tester ou les optimiser autant que je l'aurais souhaité.

J'ai consacré beaucoup de temps à revoir continuellement les mêmes fonctions. Au lieu de pouvoir avancer progressivement vers de nouvelles fonctionnalités, j'ai souvent dû revenir sur mon implémentation initiale afin de corriger des problèmes ou ajuster certains comportements après des tests échoués. J'ai également passé beaucoup de temps à lire, comprendre et intégrer le travail des autres membres afin d'assurer la compatibilité entre toutes les parties du projet.

En fin de semestre, j'ai pu travailler davantage sur l'intégration et la dockerisation de l'application, deux aspects que je souhaitais apprendre. Cette partie a été très formatrice et m'a permis de comprendre le déploiement et la structuration d'une application réelle.

Globalement, je suis satisfait du résultat final et du chemin parcouru. Malgré les difficultés, nous avons réussi à produire une application fonctionnelle et cohérente. Néanmoins, j'ai parfois eu l'impression de consacrer davantage d'énergie à l'organisation et à la coordination du projet qu'à l'exploration technique approfondie de certaines fonctionnalités que j'aurais aimé développer davantage. Bien que le groupe soit satisfait de mon leadership, mon bilan personnel reste teinté d'un léger regret de ne pas avoir poussé les fonctionnalités et l'application plus loin et de me concentrer pleinement sur mon code par moments.

Avec du recul, si l'expérience était à refaire, je modifierais deux aspects méthodologiques :

1. **Prolonger la phase d'étude technologique** : Face à la multitude d'outils disponibles dans le domaine de l'IA, nous aurions dû prendre plus de temps pour les tester en profondeur avant d'arrêter nos choix.
2. **Adopter le TDD (Test-Driven Development)** : Baser notre développement sur des tests unitaires rédigés en amont de l'écriture du code aurait grandement facilité l'intégration et limité les réajustements tardifs.

Enfin, cette expérience m'a permis de développer une compréhension beaucoup plus large des modèles de langage et des technologies d'intelligence artificielle actuelles. Au fil des recherches et du développement, j'ai développé un réel intérêt pour le fonctionnement et l'évolution de ces systèmes. Même si le rôle de chef de projet a représenté une charge importante tout au long du semestre, il m'a toutefois permis de connaître précisément chaque force et chaque faiblesse de notre application, et de progresser autant techniquement qu'humainement.

5 Débat sur l'IA Générative

5.1 Transparence : utilisation des outils d'IA générative

Tout au long du projet, l'équipe a utilisé plusieurs outils d'IA générative comme ChatGPT, Gemini, Claude, Pleiade. Leur utilisation a principalement servi d'aide à la compréhension, à la recherche d'informations techniques, à l'organisation du travail et à l'accélération de certaines tâches de développement.

Ces outils nous ont notamment permis de mieux comprendre certains concepts complexes liés au projet, comme les embeddings, la similarité cosinus, les bases de données vectorielles ou encore le fonctionnement des serveurs d'inférence et des modèles de langage. Ils ont également été utiles pour le débogage, l'explication d'erreurs, la découverte de bibliothèques Python, ainsi que pour la rédaction de certaines parties de documentation et de rapports, notamment en $\text{\LaTeX}$.

L'IA générative a aussi joué un rôle important dans l'organisation générale du projet. Elle nous a aidés à structurer certaines idées, établir des plans de travail, vérifier que certains objectifs techniques étaient atteints ou encore comparer différentes approches d'implémentation.

Avec le temps, nous avons également constaté les limites de ces outils. Lorsque le projet est devenu plus complexe et que la base de code a grandi, il devenait de plus en plus difficile

d'expliquer précisément le contexte complet aux modèles d'IA. Les réponses devenaient alors incomplètes, imprécises ou mal adaptées à notre architecture. Cette expérience nous a rapidement montré qu'il était impossible de se reposer entièrement sur ces outils sans comprendre réellement les choix techniques effectués.

5.2 Analyse critique : apport réel des outils et limites pédagogiques

L'utilisation de l'IA générative nous a apporté un gain de productivité important, notamment en réduisant le temps passé à rechercher des informations dans les documentations officielles ou sur différents forums techniques, ou d'obtenir des exemples d'utilisation de bibliothèques ou des pistes de correction pertinentes. Cela nous a permis d'avancer plus rapidement, surtout lors des phases de blocage ou lorsqu'il fallait produire du code dans des délais courts.

Cependant, cette rapidité peut aussi donner l'impression que les solutions sont simples et immédiatement applicables, alors qu'en pratique une grande partie du travail consiste justement à comprendre, adapter, tester et corriger ce qui est proposé.

Nous avons rapidement remarqué qu'un code généré automatiquement pouvait être techniquement correct tout en étant inadapté au projet ou à notre architecture. Copier du code sans le comprendre est contre-productif.

Au lieu de considérer l'IA comme une solution automatique, nous l'avons progressivement utilisée comme un assistant de réflexion, et le véritable apprentissage venait surtout du travail d'analyse, des tests, de la correction des erreurs et de la compréhension des concepts sous-jacents.

Le fait de développer nous-mêmes un assistant pédagogique basé sur des modèles de langage nous a également permis de mieux comprendre les limites actuelles de ces technologies.

Les modèles peuvent halluciner, manquer de contexte ou produire des réponses crédibles mais incorrectes. Ces problématiques rejoignent directement celles que nous avons cherché à traiter dans notre propre système, notamment à travers les mécanismes de filtrage et de contextualisation des réponses.

Finalement, nous considérons que l'IA générative constitue un excellent outil d'accompagnement et d'accélération du travail, mais qu'elle ne remplace ni la compréhension technique, ni la réflexion, ni l'apprentissage réel nécessaires à la réalisation d'un projet complexe.

6 Compétences Acquises

6.1 Compétences techniques

Au cours du projet, l'équipe a développé plusieurs compétences techniques liées au développement d'applications basées sur l'intelligence artificielle et les modèles de langage.

Le projet a notamment permis de découvrir et de mettre en pratique les principes des systèmes de RAG (*Retrieval-Augmented Generation*), incluant :

- l'ingestion et le traitement de documents ;
- le découpage en fragments (*chunks*) ;
- la génération de représentations vectorielles (*embeddings*) ;
- l'indexation dans une base de données vectorielle ;
- la recherche sémantique et le calcul de similarité.

L'équipe a également acquis une meilleure compréhension du fonctionnement des modèles de langage locaux, des serveurs d'inférence ainsi que des contraintes liées au contexte et aux performances des LLM et SLM.

Sur le plan back-end, le projet a permis de travailler avec :

- FastAPI ;
- PostgreSQL ;
- SQLAlchemy ;
- pgvector ;
- Ollama ;
- pytest et unittest ;
- Docker pour la conteneurisation et le déploiement.

Le développement du système de filtrage a aussi permis d'acquérir des compétences en *prompt engineering*, en évaluation de modèles et en conception de mécanismes de validation des réponses générées par l'IA.

Concernant le front-end, l'équipe a appris à développer une interface moderne avec React et Vite, à gérer les appels API ainsi qu'à assurer l'intégration entre les différentes parties de l'application.

Enfin, le projet a permis de renforcer plusieurs bonnes pratiques de développement logiciel, notamment :

- la modularité du code ;
- l'intégration entre plusieurs composants ;
- l'écriture de tests ;
- le débogage ;
- la maintenance et l'amélioration continue d'une base de code existante.

6.2 Compétences méthodologiques

Au-delà des aspects techniques, ce projet a également permis de développer plusieurs compétences méthodologiques et organisationnelles. Le travail en équipe sur un projet de longue durée a mis en évidence l'importance :

- de la communication ;
- de la répartition des tâches ;
- du suivi d'avancement ;
- de la documentation ;
- et de la coordination entre les différents modules du projet.

L'équipe a également appris à travailler avec des contraintes réelles de développement, notamment :

- des délais limités ;

- des difficultés d'intégration ;
- des changements techniques en cours de projet ;
- ainsi que des imprévus organisationnels.

Le projet a aussi sensibilisé l'équipe à l'importance des phases d'étude technologique, du choix des outils adaptés ainsi que de l'intérêt des tests unitaires et des méthodologies de développement structurées comme le *Test-Driven Development* (TDD).

Enfin, cette expérience a permis de développer une meilleure capacité d'adaptation, d'autonomie et de résolution de problèmes face à des situations techniques complexes.

7 Difficultés rencontrées et solutions apportées

Lors de la réalisation de ce projet, nous avons rencontré plusieurs difficultés, aussi bien techniques qu'organisationnelles. Voici les principaux obstacles auxquels nous avons fait face et les solutions que nous avons mises en place pour les surmonter :

— Tests en local et performances

Difficulté : Lors du développement, les tests du pipeline d'ingestion en local prenaient énormément de temps (souvent plusieurs dizaines de minutes). Au début, nous pensions que cette lenteur était simplement due à la complexité des modèles d'intelligence artificielle tournant sur nos ordinateurs personnels.

Solution : Lorsque nous avons conteneurisé l'application avec Docker. Nous avons compris après une analyse de l'utilisation des ressources que le problème de RAM venait en réalité de la bibliothèque *Marker*. Nous l'avons donc remplacée par *Docking*, ce qui a considérablement allégé la mémoire et rendu nos tests beaucoup plus rapides.

— Paramétrage du modèle de filtrage

Difficulté : La configuration du petit modèle de langage (qui fait office de "juge") a été plus complexe que prévu. Il était difficile de le forcer à répondre uniquement par OUI ou NON ; il avait souvent tendance à générer des phrases complètes ou à ne pas respecter nos consignes strictes.

Solution : Nous avons réalisé de nombreux tests comparatifs avec différents modèles (Llama, Phi, Ministral) et ajusté nos prompts de manière itérative. Après plusieurs essais, nous avons trouvé le modèle et la formulation adéquats pour obtenir un filtre stable et précis.

— Fidélité de l'interface graphique

Difficulté : Les premières solutions que nous avons testées pour le front-end manquaient de flexibilité. Elles ne nous permettaient pas de reproduire fidèlement l'organisation des pages et le design des maquettes que nous avions validés avec notre encadrant.

Solution : Nous avons donc décidé de changer d'approche en cours de projet pour utiliser le framework React. Ce choix nous a offert la liberté nécessaire pour construire l'interface exactement comme nous l'avions imaginée, en séparant bien l'espace étudiant de l'espace enseignant.

— Intégration Front-end / Back-end

Difficulté : Connecter les différentes parties du projet (l'interface d'un côté, l'API FastAPI et le système RAG de l'autre) a généré des erreurs inattendues. Nous avons eu des problèmes de communication entre les modules, notamment pour l'envoi des questions, l'affichage de l'historique ou la gestion des documents.

Solution : Au lieu de tout intégrer en une seule fois, nous avons adopté une méthode progressive. Nous avons testé et relié chaque route de l'API une par une avec le front-end, ce qui nous a permis d'identifier et de corriger les erreurs de liaison au fur et à mesure.

— Organisation et travail d'équipe

Difficulté : Avec des emplois du temps différents et quelques imprévus durant le semestre, il a été difficile de trouver des créneaux pour travailler tous ensemble. Ce manque de synchronisation a parfois ralenti notre prise de décision.

Solution : Nous nous sommes adaptés en privilégiant une communication régulière et une répartition claire des tâches. Un membre du groupe a pris le rôle de coordinateur pour faciliter les décisions. Pour rester alignés, nous nous sommes fortement appuyés sur nos maquettes et le cahier des charges, ce qui nous a permis d'avancer de manière autonome tout en gardant une vision commune.

8 Pistes d'Amélioration

AP-UP constitue une base solide, mais plusieurs axes d'amélioration peuvent être envisagés pour faire évoluer le projet vers une solution encore plus complète.

— Sur le plan de l'architecture

Le système a été conçu de manière modulaire dès le départ, ce qui facilite l'intégration de nouveaux modèles ou outils sans avoir à restructurer l'ensemble du projet. À terme, le passage à des modèles plus puissants hébergés sur Pléiade permettrait d'améliorer la qualité des réponses générées et la fiabilité du filtrage. Un système de cache pourrait également être mis en place pour accélérer les réponses sur des questions déjà posées, et le déploiement pourrait être étendu à plusieurs universités ou départements partageant la même infrastructure.

— Sur le plan des fonctionnalités

L'intégration avec des plateformes universitaires existantes comme Moodle représente une évolution naturelle du projet. Elle permettrait aux enseignants de gérer leurs documents directement depuis leur environnement habituel, sans passer par un *dashboard* séparé. Un système de *feedback* étudiant pourrait aussi être ajouté pour permettre à l'étudiant de signaler une réponse incorrecte ou insatisfaisante, ce qui contribuerait à améliorer progressivement la qualité du système.

— Sur le plan du suivi et de la personnalisation

Un tableau de bord de statistiques permettrait aux enseignants de visualiser les questions les plus fréquentes, les réponses bloquées et l'utilisation globale de l'assistant par module. Le module de filtrage pourrait également être enrichi par des

métriques automatiques mesurant le taux de questions acceptées et bloquées, afin d'ajuster les seuils de manière empirique. Il serait aussi intéressant de permettre à l'enseignant de personnaliser ces seuils directement depuis l'interface, selon le niveau de rigueur souhaité pour chaque cours.

9 Conclusion

Dans ce projet, nous avons développé AP-UP, un assistant pédagogique conversationnel permettant aux étudiants de poser des questions sur leurs cours, en s'appuyant uniquement sur les documents fournis par les enseignants. L'objectif était de créer un outil utile, accessible et adapté au contexte universitaire.

Aujourd'hui, AP-UP répond aux principales attentes du cahier des charges. L'application permet aux étudiants d'utiliser un espace de *chat* et aux enseignants de gérer les documents pédagogiques utilisés par l'assistant. Même si des améliorations restent possibles, nous avons réussi à produire une base fonctionnelle et cohérente.

Ce projet a aussi montré l'intérêt potentiel d'un tel outil dans un contexte universitaire. AP-UP pourrait aider les étudiants à mieux exploiter leurs supports de cours, tout en donnant aux enseignants un moyen de contrôler les documents utilisés par l'assistant. Avec davantage de temps, l'application pourrait être améliorée sur certains aspects, comme l'ergonomie, la gestion avancée des utilisateurs ou l'export des réponses.

Globalement, ce projet a été une expérience enrichissante, à la fois sur le plan technique et organisationnel. Il nous a permis de progresser, de gagner en autonomie et de mieux comprendre les étapes nécessaires à la conception d'une application complète intégrant de l'intelligence artificielle.

Glossaire

AP-UP (Assistant Pédagogique Université Paris Cité) Application conversationnelle locale développée dans ce projet, permettant aux étudiants d'interroger un assistant intelligent strictement limité aux documents pédagogiques déposés par leurs enseignants.

Architecture RAG (Retrieval-Augmented Generation) Architecture combinant une phase de recherche documentaire et une phase de génération de texte : le système récupère d'abord les passages pertinents dans une base de documents, puis les fournit au modèle de langage pour produire une réponse sourcée.

Base de données vectorielle Base de données spécialisée dans le stockage et la recherche de vecteurs numériques, permettant des recherches par similarité sémantique. Dans AP-UP, PostgreSQL avec l'extension pgvector remplit ce rôle.

Chunk (fragment) Portion de texte obtenue après découpage d'un document original. Chaque chunk est transformé en vecteur et stocké pour permettre la recherche sémantique.

Embedding (représentation vectorielle) Représentation numérique d'un texte sous forme d'un vecteur de nombres réels. Ces vecteurs permettent de calculer mathématiquement la ressemblance sémantique entre une question et les passages d'un document.

Hallucination Phénomène par lequel un modèle de langage génère des informations incorrectes ou inventées tout en les présentant avec confiance. AP-UP cherche à réduire ce risque en contraignant le modèle à s'appuyer uniquement sur les documents fournis.

LLM (Large Language Model) Modèle d'intelligence artificielle de grande taille capable de comprendre et générer du langage naturel. Dans AP-UP, le LLM est exécuté localement via Ollama sur le serveur Pléiade.

Ollama Outil permettant d'exécuter localement des modèles de langage open source sur les serveurs de l'université, sans recourir à des services cloud externes, garantissant la confidentialité des données.

pgvector Extension de PostgreSQL permettant de stocker des vecteurs numériques et d'effectuer des recherches par similarité directement dans la base de données relationnelle.

Pléiade Serveur de l'Université Paris Cité hébergeant AP-UP et exécutant localement les modèles de langage, garantissant que les données pédagogiques ne transitent pas par des services tiers.

Recherche sémantique Méthode de recherche basée sur le sens des mots plutôt que sur la correspondance exacte de mots-clés, reposant sur la comparaison de vecteurs via la similarité cosinus.

Similarité cosinus Mesure mathématique évaluant la ressemblance entre deux vecteurs. Utilisée dans AP-UP pour identifier les passages les plus proches d'une question et valider la pertinence des réponses générées.

SLM (Small Language Model) Modèle de langage de taille réduite, plus léger et rapide qu'un LLM. Dans AP-UP, le modèle ministral-3 :3b joue le rôle de « juge » pour filtrer les requêtes hors-sujet.

Références

Concepts et Modèles d'Intelligence Artificielle

- [1] IBM Technology (2023). *What is Retrieval-Augmented Generation (RAG) ?*. Vidéo explicative sur le concept central de l'architecture RAG.
Disponible sur : https://youtu.be/T-D10fcDW1M?si=xUlvnz4vSrX0_eo
- [2] Ollama. Plateforme d'exécution locale ayant servi de source pour tous les modèles de langage testés et utilisés dans le projet (notamment Ministral-3 :3b, phi4-mini, etc.).
Disponible sur : <https://ollama.com/>
- [3] OpenAI Whisper. Modèle de transcription automatique utilisé pour le traitement des fichiers audio et vidéo.
Disponible sur : <https://github.com/openai/whisper>
- [4] Hugging Face. Modèle d'embedding (intfloat/multilingual-e5-large) utilisé pour la vectorisation sémantique.
Disponible sur : <https://huggingface.co/intfloat/multilingual-e5-large>

Développement et Traitement des données

- [5] Docling (IBM). Outil d'extraction et de conversion en Markdown pour la gestion des documents PDF complexes.
Disponible sur : <https://github.com/docling-project/docling>
- [6] Unstructured. Bibliothèque utilisée pour le parsing documentaire et le traitement des documents scannés via OCR.
Disponible sur : <https://docs.unstructured.io/>
- [7] pdfplumber. Bibliothèque d'extraction de texte légère pour les PDF strictement textuels.
Disponible sur : <https://github.com/jsvine/pdfplumber>
- [8] Sentence-Transformers. Bibliothèque de génération d'embeddings vectoriels pour la recherche sémantique.
Disponible sur : <https://sbert.net/>
- [9] pgvector. Extension PostgreSQL pour le stockage et la recherche par similarité vectorielle.
Disponible sur : <https://github.com/pgvector/pgvector>
- [10] FastAPI. Framework Python utilisé pour la création de l'API REST servant de back-end.
Disponible sur : <https://fastapi.tiangolo.com/>
- [11] React. Bibliothèque JavaScript utilisée pour le développement de l'interface utilisateur web.
Disponible sur : <https://react.dev/>
- [12] pytest. Documentation officielle du framework utilisé pour les tests unitaires en Python.
Disponible sur : <https://docs.pytest.org/>

Infrastructure, Tutoriels et Outils d'aide au projet

- [13] La Forge - Documents du projet. Plateforme institutionnelle regroupant les documents de suivi du projet L3S (cahier des charges, cahier de recettes, dossiers de conception, etc.).
Disponible sur : <https://forge.ens.math-info.univ-paris5.fr/projects/2025-13s1/documents>
- [14] Pléiade. Serveurs de l'université utilisés pour l'hébergement et l'exécution sécurisée des modèles locaux.
Disponible sur : <https://pleiade.mi.parisdescartes.fr/>
- [15] Docker. Technologie de conteneurisation pour l'optimisation des performances en local et le déploiement.
Disponible sur : <https://www.docker.com/>
- [16] Lucidchart & PlantUML. Outils de modélisation utilisés pour la création des diagrammes d'architecture UML.
Disponible sur : <https://www.lucidchart.com/> / <https://plantuml.com/>
- [17] Overleaf. Éditeur LaTeX utilisé pour la mise en page technique et la production du rapport final.
Disponible sur : <https://www.overleaf.com/>
- [18] Google Gemini & Claude. Assistants IA utilisés pour l'aide à la compréhension technique, le débogage et la structuration de la documentation.
Disponible sur : <https://gemini.google.com/> / <https://claude.ai/>
- [19] PGVector : Turn PostgreSQL Into A Vector Database. Tutoriel YouTube pour l'implémentation de la base de données vectorielle.
Disponible sur : <https://youtu.be/j1QcPSLj7u0>

Index

- API, 2, 6, 12, 16, 20, 22, 25
- API / FastAPI, 9
- Architecture modulaire, 2, 10
- Assistant Pédagogique (AP-UP), 2, 3, 13, 14, 22–24
- Base de données vectorielle, 6, 8, 10, 12, 15, 17, 18, 20, 24, 26
- Cahier de recettes, 3, 26
- Cahier des charges, 3, 5, 22, 23, 26
- Chainlit, 6, 16
- Chainlit / React / Vite, 9, 10
- Chunk, 15, 20, 24
- Chunking, 3, 5, 6, 11
- Diagramme de Gantt, 13
- Docker, 10, 17, 20, 21, 26
- Embeddings, 15, 18, 20, 24, 25
- Embeddings (Vectorisation), 5–8
- Extraction, 25
- Extraction de données, 3, 5, 8, 10
- FastAPI, 12, 16, 20, 22, 25
- Filtrage, 2, 4, 6, 9–12, 15, 19–22
- Forge, 14, 26
- Fragment, 20, 24
- Hallucination, 2, 19, 24
- Livable, 3, 14
- llama3.2, 12, 15
- llama3.2 / phi4 / ministral-3 :3b, 9, 10
- LLM, 2, 3, 10, 11, 20, 24
- LLM (Large Language Model) / SLM (Small Language Model), 6, 8
- Marker, 11, 21
- ministral-3 :3b, 11, 12, 24, 25
- Moodle, 22
- Métadonnées, 11, 15
- Méthodes agiles, 15
- Ollama, 6, 17, 20, 24, 25
- Ollama / OpenAI, 8
- OpenAI, 25
- ORM, 11
- ORM / SQLAlchemy, 8
- Parsing, 15, 25
- pdfplumber, 10, 25
- pgvector, 6, 11, 20, 24, 25
- pgvector / PostgreSQL, 8
- phi4, 12, 25
- Pléiade, 3, 6, 8, 10
- PostgreSQL, 11, 20, 24–26
- Prompt système, 5, 9, 11, 15
- Propriété intellectuelle, 2
- pytest, 9, 20, 25
- Python, 8, 18, 25
- RAG, 2, 4, 8, 10–12, 16, 17, 20, 22, 24, 25
- React, 12, 16, 20, 21, 25
- RGPD, 2
- Scraping / Playwright / Whisper / Marker / PDFPlumber, 8, 9
- Similarité cosinus, 8, 11, 12, 15, 18, 24
- SLM, 12, 15, 20, 24
- SQLAlchemy, 11, 20
- SVN, 14
- TDD, 18, 21
- Tests (unitaires), 9, 12, 13
- Tests unitaires, 3, 15, 18, 21, 25
- UML, 3, 26
- Vectorisation, 3, 25
- Vite, 16, 20
- Whisper, 10, 25